

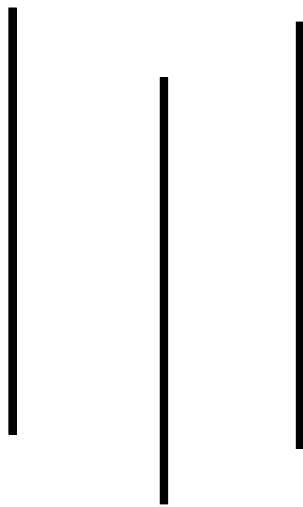
Multiple View Geometry

Lab4 –Stereo Visual Odometer

Lab Sessions: 2 (4 hours)

Total Work Load: 10 hours

Programming Language: MATLAB



Submitted by

1. **Pravin Oli** (u1999699) [*pravin.oli.08@gmail.com*]
2. **Gebrecherkos Gebrezgabhier** (u1999542) [*chereg2016@gmail.com*]

Table of Contents

1.Objectives:	3
2.Methodology:	3
Part-1 (Data Setup)	3
Information About Data Set:.....	3
Camera Specifications:	3
Dataset Description	3
Part-2 (Implementation).....	4
Camera Parameter Setup:.....	4
Processing new incoming frame:	4
Feature Extraction:	4
Circular Matching:	5
Compare two methods to estimate stereo motion:	6
Performing sensitivity analysis using Monte Carlo:	7
Processing the complete sequence of stereo pairs:	8
3.Questions.....	8
Monte Carlo analysis	9
Processing the sequence.....	9
4.Conclusion	10

1.Objectives:

- To compare implementation of visual odometers and visual SLAM for outdoors robots.
- To Implement a Monte Carlo method to analyse and visualize the uncertainty
- Apply the simple odometer for a larger sequence of stereo images

2.Methodology:

Part-1 (Data Setup)

Information About Data Set:

Dataset Name: UTIAS Long-Term Localization and Mapping Dataset

Source: Collected using a Clearpath Robotics Grizzly Robotic Utility Vehicle with a Point Grey Bumblebee XB3 stereo camera.

Camera Specifications:

Type: Point Grey Bumblebee XB3 stereo camera.

Baseline: 0.24 meters.

Resolution: 512 x 384 pixels.

Frame Rate: Captured at 16 Hz, but downsampled to provide stereo keyframes approximately every 0.2 meters traveled.

Dataset Description

Environment: Mix of buildings, fencing, roads, and off-road vegetation.

Data Collection: Includes 39 runs over a 260-meter path. Data collected approximately every hour over 30 hours.

Number of Stereo Pairs: 981 stereo image pairs.

Dataset accessed from the University of Toronto FTP server.

Implementation:

The stereo images are organized into two separate folders (images/left/ and images/right/) for the left and right camera streams, respectively. These images are loaded into two MATLAB imageDatastore objects, imdsLeft and imdsRight, which provide easy access to the image sequences. Finally, the script reads and displays the first pair of stereo images side by side using imshowpair, allowing for visual inspection of the dataset.

Output:

First stereo image pair of the sequence



Part-2 (Implementation)

Camera Parameter Setup:

The intrinsic parameters for camera1 and camera2 are equal and the extrinsic parameters of camera2 (right camera) is defined by the baseline and no rotation in the coordinate system.

$$K = \begin{bmatrix} f_u & 0 & u_0 \\ 0 & f_v & v_0 \\ 0 & 0 & 1 \end{bmatrix} \text{ and } P = K [R \quad t]$$

`initialPoseRotation` defines the orientation of the camera relative to the vehicle.

`initialPoseTranslation` specifies the position of the camera relative to the vehicle in meters.

`rigid3d Object` Combines rotation and translation to represent the initial 3D pose of the camera.

Focal Length: [387.777, 387.777] in pixels.

Principal Point: [257.446, 197.718] in pixels.

Intrinsic Matrix: A 3x3 matrix representing the camera's internal geometry

Image Size: Extracted from the dimensions of the left stereo image.

Baseline: 0.239965 meters, representing the distance between the left and right cameras.

Rotation: Identity matrix (no rotation between the cameras).

Translation: [-baseline, 0, 0] to specify the offset between cameras.

`stereoParameters Object`: Combines parameters of both cameras into a single structure for stereo calibration and processing.

Processing new incoming frame:

This pre-processing step prepares stereo images for further processing, ensuring they are free from distortions and aligned for accurate depth computation and feature matching.

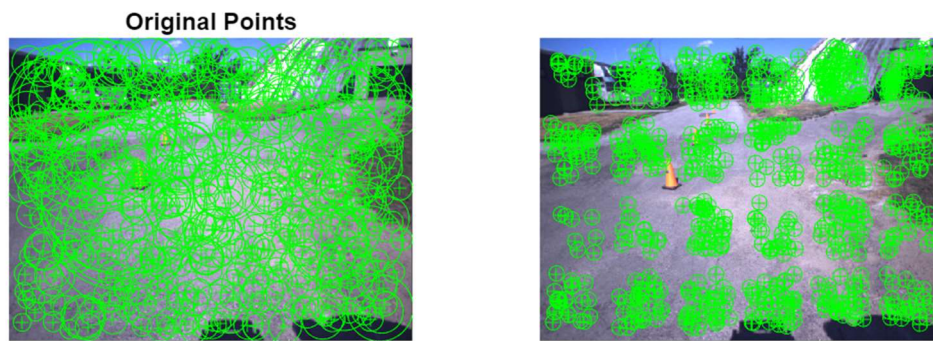
Output:



Feature Extraction:

After pre-processing the stereo images (undistortion and rectification), the next step is to extract features for stereo matching. Feature detectors typically work on the entire image, which can lead to problems like Clustering of features in areas with high texture (e.g., edges or corners), and Sparse or no features in low-texture areas (e.g., walls, open sky).

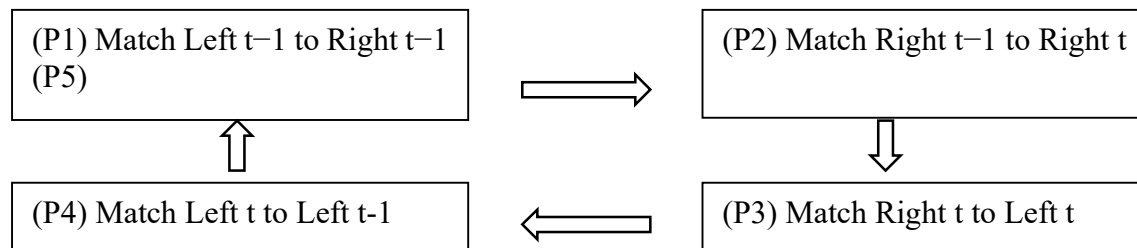
This problem is solved by bucketing. It works by dividing the image into a grid (e.g., uniform rectangular "buckets") and then extracting features independently from each grid cell to ensure a more uniform distribution of features across the image.



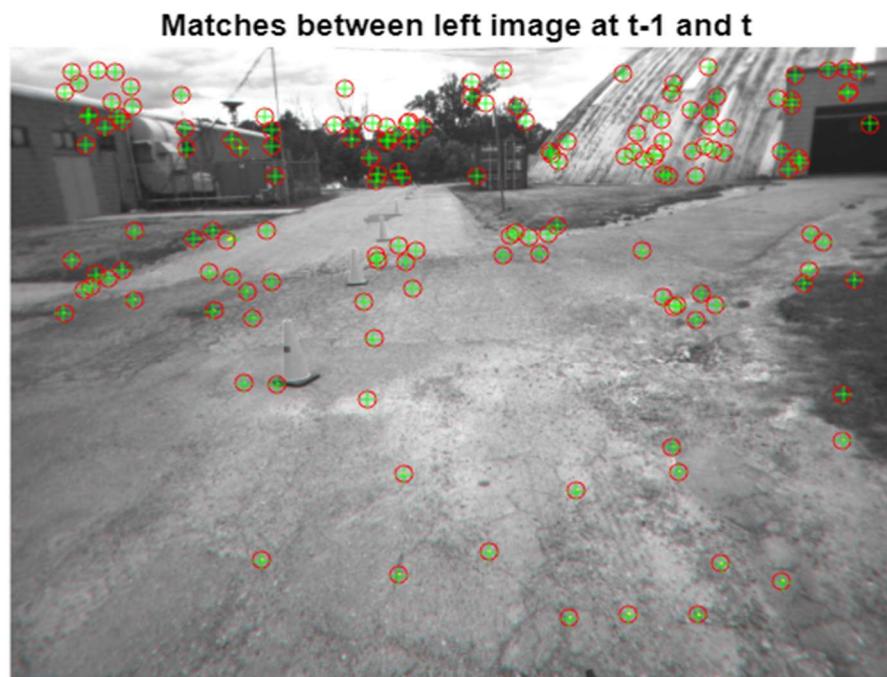
Circular Matching:

In this matching technique features P1 from left camera is compared with features from right camera at time $t-1$, and best match features are stored as feature P2. This process is repeated in loop until $P5=P1$ which is considered as good match.

Process:



Output:

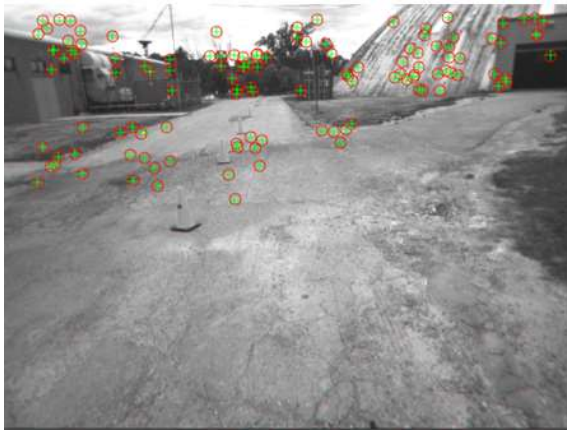
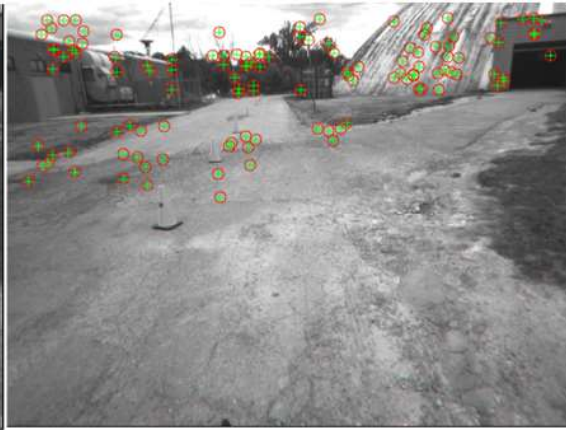
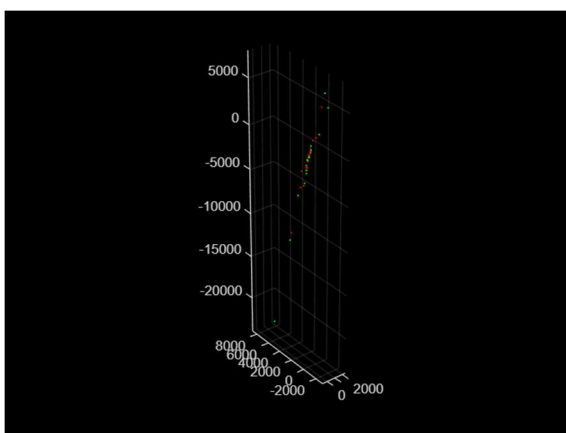
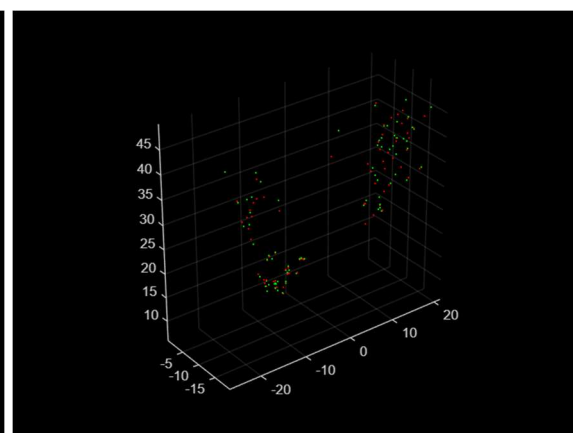


Compare two methods to estimate stereo motion:

3D-to-3D motion estimation aligns two 3D point clouds (sets of 3D points) obtained at consecutive time steps. It computes the rotation matrix $[R]$ and translation vector $[t]$ that best aligns the 3D points from time $t-1$ to the 3D points at time t .

2D-to-3D motion estimation uses 2D keypoints from the current frame (time t) and 3D points from the previous frame (time $t-1$) to compute the camera pose (rotation R and translation t) relative to the previous frame.

S. N	3D to 3D	2D to 3D
1	Point cloud alignment	Use Perspective-n-Point (PnP)
2	Works with 3D-3D correspondences	Works with 2D-3D correspondences
3	No need for camera intrinsic	Requires intrinsic matrix K
4	More sensitive to depth noise	More robust to depth noise
5	Requires large 3D point clouds	Works with fewer points

Output:*Figure 1 3D to 3D Left Matched**Figure 2 3D to 3D Right Matched**Figure 3 Two points clouds**Figure 4 Two points clouds after removing invalid area***Output:**

The result of 3D to 3D (translation) is	-0.048693	-0.011978	-0.081944
The result of 2D to 3D (translation) is	0.0022563	-0.013716	-0.010787

Performing sensitivity analysis using Monte Carlo:

The Monte Carlo sensitivity analysis is implemented to evaluate the robustness of 3D to 3D and 2D to 3D motion estimation methods under noisy conditions.

Here

Standard deviation = 0.1

Number of Runs = 1000

Implementation: Monte Carlo Sensitivity Test: 3D-to-3D vs. 2D-to-3D**Noise Addition:**

Add Gaussian noise to the 2D matched points (left1, right1, left2, right2)

Noise Addition:

Add Gaussian noise to the 2D matched points (left1, right1, left2, right2)

3D Triangulation:

Compute 3D points at t-1 and t using noisy 2D matches.

3D Triangulation:

Compute 3D points at t-1 and t using noisy left1 and right1.

Point Filtering:

Remove 3D points outside a valid Z-range ($z_{min}=0.5$, $z_{max}=50$).

Camera Pose Estimation:

Use noisy left2 points and triangulated 3D points (t-1) to solve PnP and estimate R and t.

Rigid Transformation:

Align noisy 3D point clouds and estimate rotation (R) and translation (t) using rigid transformation.

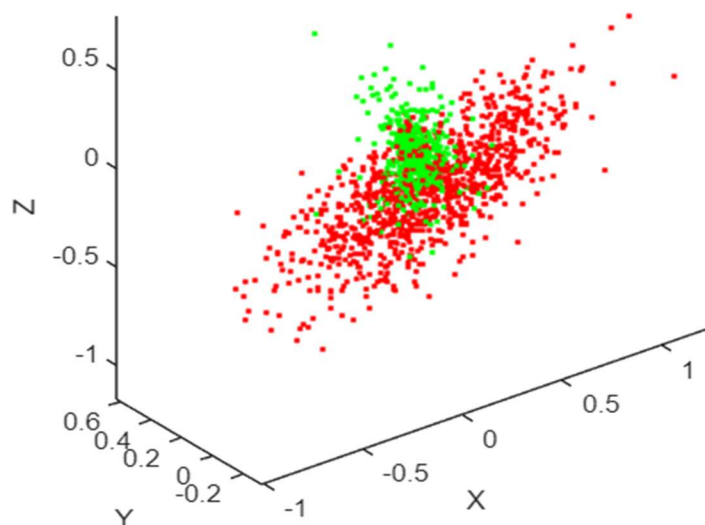
Output:

Figure 5 Sensitivity analysis using Montecarlo

Processing the complete sequence of stereo pairs:

In this step sequence of stereo image pairs are processed to estimate the motion (Translation and Rotation) of the camera over time and builds the trajectory.

GPS Trajectory:

In this step GPS trajectory in world reference frame is plotted.

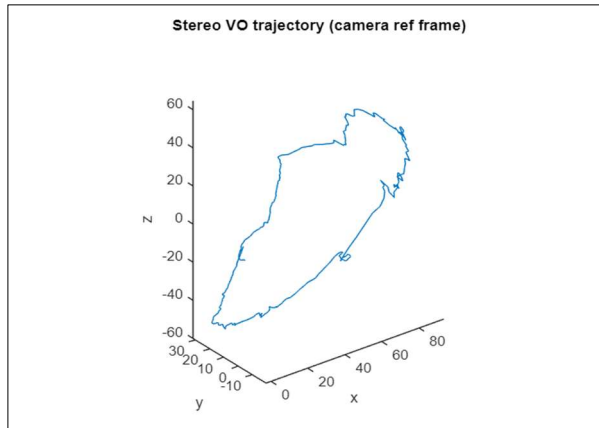


Figure 6 Stereo VO Trajectory

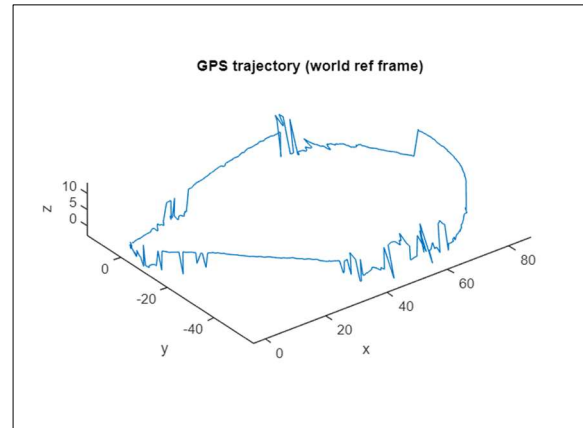
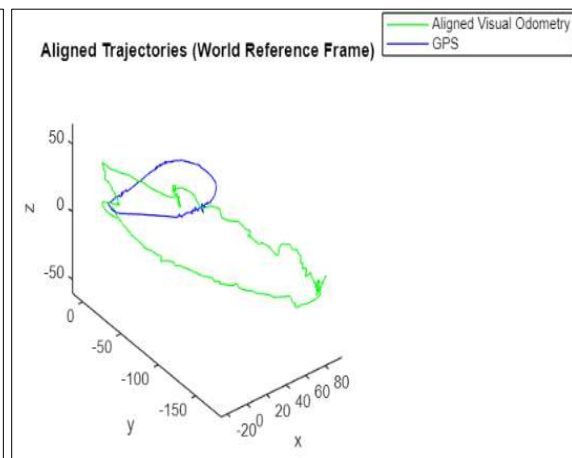
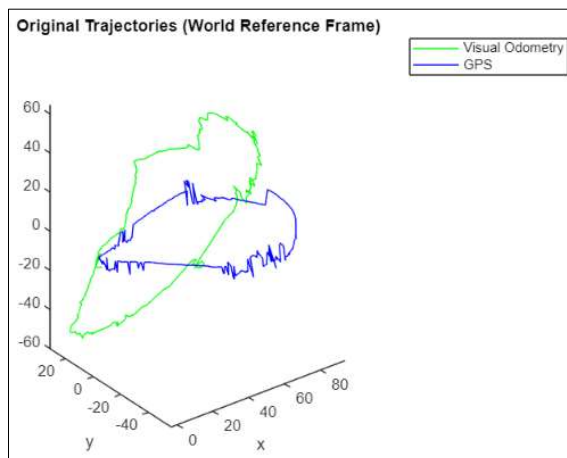


Figure 7 GPS Trajectory

Align the two trajectories and then plot the two trajectories aligned:

**3. Questions****Circular matching**

a. Does the circular matching use geometric constraints?

➔ No, the basic circular matching approach does not directly impose geometric constraints.

b. Does it guarantee correct matches?

➔ No, circular matching does not guarantee correct matches, especially in scenarios with repetitive patterns or significant occlusions.

c. If not, then

- describe a situation where it would fail.

➔ some of the situations where it would fail are:

- i. Repetitive Patterns: Features in areas with repeating designs can wrongly match with multiple similar-looking points.
- ii. Parallax Effects: Large shifts in viewpoint cause features to appear differently, leading to mismatched points.
- iii. Occlusions: When an object blocks part of the view, valid matches are lost or replaced with incorrect ones.
- iv. Scale and Rotation Variations: Features that change size or orientation between frames might fail to match accurately.

- provide a detailed explanations on what geometric constraints could be imposed and how.

➔ To improve the accuracy and reliability of feature matching in circular matching, the geometric constraints are:

- i. Epipolar constraints: It ensures that corresponding points in two views lie on the epipolar line of the other image.
- ii. Triangulation check: It ensures the matched points produce valid 3D coordinates when triangulated.

Monte Carlo analysis

The 2D-to-3D method uses 3 images whereas the 3D-to-3D method uses 4. However, the 3D-to-3D method performs considerably worse. Please provide a justification on what may be the cause of this.

➔ The 3D-to-3D method performs worse primarily due to noise amplification in depth, compounded errors in triangulation, and the lack of constraints provided by the camera projection model.

In the 2D-to-3D method, Errors in depth are not directly used for motion estimation. The method relies on reprojection constraints (2D points compared to projected 3D points), which are more robust to noise. But in 3D-to-3D method, matching in 3D introduces additional degrees of freedom noise affects x, y, z coordinates unlike 2D matching which only deals with x, y. This makes 3D-to-3D more vulnerable to noise and outliers.

Processing the sequence

As you can see, the simple methods seen in this example have modest performance. How could it be improved? Define and describe additions to these methods, or alternative strategies that you consider that would improve the performance in terms of accuracy. Focus you answer on the 3D reconstruction and geometric aspects and not on the navigation and filtering aspects (such as EKF or SLAM). Use between half a page to one page for your answer.

➔ To improve the accuracy of 3D reconstruction and motion estimation in visual odometry, the following strategies can be adopted:

- a. Robust Feature Detection and Matching

Using advanced feature descriptors like ORB or Super Point for more robust matching, and applying bidirectional matching and geometric constraints (e.g., epipolar lines) to reduce mismatches.

- b. Improved Triangulation

Using weighted triangulation to prioritize matches with higher confidence and reject outliers based on reprojection errors.

c. Refined 3D Point Clouds

Filtering noisy points using outlier removal techniques and enhance point density with multi-view stereo methods.

d. Enhanced Calibration

Refining intrinsic parameters with online calibration continuously and making align stereo baselines with additional reference points.

e. Use of Scene Priors

Utilizing semantic information (e.g., detected objects or ground planes) to enforce geometric consistency and guide motion estimation.

4. Conclusion

In this lab we successfully implemented and compared 3D-to-3D and 2D-to-3D motion estimation methods for visual odometry, evaluated using Monte Carlo sensitivity analysis, and processed stereo image sequences to reconstruct camera trajectories. The results showed that 2D-to-3D motion estimation, relying on 2D-3D correspondences and Perspective-n-Point (PnP), is more robust to noise and depth errors compared to the 3D-to-3D approach, which directly aligns noisy 3D point clouds. The trajectories estimated through visual odometry were aligned with GPS data using geometric transformations, demonstrating the effectiveness of the implemented methods. By using techniques such as feature bucketing, circular matching, and motion estimation, this lab provided insights into improving visual odometry accuracy.